

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 典型混沌系统方程求解

学院: 计算机与软件学院

专业: 人工智能卓越班

指导教师: 吴晓群、卯丙

报告人: 邓瑞霖 学号: 2024150040 班级: 01

实验时间: 2026 年 7 月 2 日 -- 2026 年 7 月 13 日

实验报告提交时间: 2026 年 7 月 3 日星期五

教务部制

实验目的与要求:

1. 灵活运用常微分方程数值解方法对复杂非线性微分方程进行求解, 增加对复杂系统中的非线性现象理解, 提升问题分析深度

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

1. 常微分方程数值解法的基本思想

在科学和工程技术中经常要解常微分方程初值问题。本章主要考察一阶微分方程初值问题

$$\frac{dy}{dx} = f(x, y(x)), \quad x \in [a, b], \quad y(x_0) = \eta$$

在常微分方程课程中已经知道, 只要 $f(x, y) \in C([a, b] \times \mathbb{R})$, 且关于 y 满足 Lipschitz 条件

$$|f(x, y) - f(x, \tilde{y})| \leq L|y - \tilde{y}|$$

则方程存在唯一解 $y = f(x)$ 。理论上虽已证明在适当条件下其解的存在性, 但在求解许多实际问题时, 仍十分困难, 甚至对一些非常简单的问题也无法用解析方法求解。为此需要介绍通用的微分方程数值解法。

所谓数值解法, 就是求 $y = f(x)$ 在一系列离散节点

$$x_0 < x_1 < \cdots < x_n < x_{n+1} < \cdots$$

上的近似值 $y_0, y_1, \cdots, y_n, y_{n+1}, \cdots$ 。约定: 相邻两个节点的距离 $h_n = x_{n+1} - x_n$ 称为

步长, 如没有特殊说明, 总假定节点等距, $h_i = h (i = 1, 2, \cdots)$ 为定值, $x_n = x_0 + nh, y_n \approx$

$y(x_n)$ 。

初值问题数值解法的基本思想是: 采用“离散化”、“步进式”, 即求解过程依节点排列的次序一步一步地向前推进。描述这类算法, 只要给出用已知信息 $y_n, y_{n-1}, \cdots$ 计算 y_{n+1} 的递推公式。

对方程离散化, 建立求数值解的递推公式, 一般有两种做法: 一种是计算 y_{n+1} 时只用

到前一点的值 y_n ，称为**单步法**。另一类是用到 y_{n+l} 前面 k 点的值 $y_n, y_{n-1}, \dots, y_{n-k+1}$ ，称为 **k (多) 步法**。

2. 数值解法的构造途径

2.1 差商逼近法

差商逼近法，即用适当差商逼近导数值使方程离散化的方法。如取

$$y'(x_n) \approx \frac{y(x_{n+l}) - y(x_n)}{h}, \quad y'(x_{n+l}) \approx \frac{y(x_n) - y(x_{n+l})}{-h}$$

引入参数 $\theta \in [0, 1]$ ，则得

$$y(x_{n+l}) \approx y(x_n) + h[\theta y'(x_n) + (1 - \theta)y'(x_{n+l})]$$

$$y_{n+l} = y_n + h[\theta f_n + (1 - \theta)f_{n+l}]$$

特别地，取 $\theta = 1$ ，则得**显式 Euler 法**

$$y_{n+l} = y_n + hf_n$$

若取 $\theta = 0$ ，则得**隐式 Euler 法**

$$y_{n+l} = y_n + hf_{n+l}$$

若取 $\theta = 1/2$ ，则得**梯形法**

$$y_{n+l} = y_n + \frac{h}{2}(f_n + f_{n+l})$$

2.2 数值积分法

数值积分方法的基本思想是将原方程转化为积分方程

$$y(x_m) - y(x_n) = \int_{x_n}^{x_m} f(x, y(x))dx, \quad y(x_0) = y_0, \quad m > n$$

然后用数值积分方法离散化，从而获得原初值问题的一个离散差分格式。

2.3 Taylor 展开法

由 Taylor 公式有

$$y(x+h) = y(x) + hy'(x) + \frac{1}{2}h^2y''(x) + \frac{1}{3!}h^3y'''(x) + \dots + \frac{1}{p!}h^py^{(p)}(x) + \frac{1}{(p+1)!}h^{p+1}y^{(p+1)}(\xi)$$

记

$$\Phi(x, y, h) = f(x, y(x)) + \frac{1}{2}hf'(x, y(x)) + \frac{1}{3!}h^2f''(x, y(x)) + \dots + \frac{1}{p!}h^pf^{(p-1)}(x, y(x))$$

可得初值问题解的近似计算公式

$$y_{n+1} = y_n + h\Phi(x_n, y_n, h), \quad n = 0, 1, \dots, N-1, \quad y_0 = \eta$$

其局部离散截断误差为

$$R_n = \frac{1}{(p+1)!} h^{p+1} y^{(p+1)}(\xi) = O(h^{p+1})$$

3. Euler 方法

在公式 $y_{n+1} = y_n + h\Phi(x_n, y_n, h)$ 中, 取 $\Phi(x, y, h) = f(x, y)$, 即得 Euler 方法 (显式格式):

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n) \\ y_0 = \eta \end{cases} \quad n = 0, 1, 2, \dots, N-1$$

N 是一个正整数, $h = \frac{b-a}{N}$ 。由上式, 从 $y_0 = \eta$ 出发, 可依次计算出 $y_1, y_2, \dots, y_N$, 它们分别为初值问题的解 $y(x)$ 在 $x_1, x_2, \dots, x_N$ 处值 $y(x_1), y(x_2), \dots, y(x_N)$ 的近似值, 其中 $x_0 = a, x_n = a + nh$ 。

由 Taylor 公式 $y(x_{n+1}) = y(x_n + h) = y(x_n) + hy'(x_n) + \frac{h^2}{2}y''(x_n) + \dots$, 对于 Euler 公式有 $y_{n+1} = y_n + hf(x_n, y_n) = y(x_n) + hy'(x_n)$, 于是

$$y(x_{n+1}) - y_{n+1} = O(h^2)$$

则 Euler 公式的截断误差为 $O(h^2)$, 所以 Euler 公式是一阶方法。Euler 方法计算量小, 但精度不高。

4. 改进的 Euler 方法 (预估-校正法)

Euler 方法计算量小, 精度不高。梯形公式

$$y_{n+1} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_{n+1}, y_{n+1})), \quad y_0 = \eta$$

的局部离散误差为 $R_n = -\frac{h^3}{12}y'''(\xi_n) = O(h^3)$, 因而是一个二阶方法。

先用 Euler 公式求得预估值 $\bar{y}_{n+1}$, 再用梯形公式进行一次校正, 得到校正值 y_{n+1} 。预估-校正公式为:

$$\begin{cases} k_1 = hf(x_n, y_n) \\ k_2 = hf(x_n + h, y_n + k_1) \\ y_{n+1} = y_n + \frac{1}{2}k_1 + \frac{1}{2}k_2 \end{cases} \quad n = 0, 1, 2, \dots, N-1$$

对于预估-校正公式, 有

$$k_1 = hf(x_n, y_n) = hy'(x_n)$$

$$k_2 = hf(x_n + h, y_n + k_1) = hy'(x_n) + h^2 y''(x_n) + \dots$$

因此 $y_{n+1} = y_n + \frac{1}{2}k_1 + \frac{1}{2}k_2 = y(x_n) + hy'(x_n) + \frac{h^2}{2}y''(x_n) + \dots$, 所以 $y(x_{n+1}) - y_{n+1} = O(h^3)$, 预估-校正公式的截断误差为 $O(h^3)$, 是二阶方法。

5. 四阶 Runge-Kutta 方法

Runge-Kutta 方法是一类高精度的一步法, 不是通过求导数的方法构造近似公式, 而是通过计算不同点的函数值, 并对这些函数值作线性组合, 构造近似公式, 再把近似公式与解的 Taylor 展开式进行比较, 使前面的若干项相同, 从而使近似公式达到一定的阶数。

在公式 $y_{n+1} = y_n + h\Phi(x_n, y_n, h)$ 中, 令

$$\Phi(x_n, y_n, h) = \sum_{i=1}^r c_i k_i$$

其中 $k_1 = f(x_n, y_n)$, $k_i = f(x_n + a_i h, y_n + h \sum_{j=1}^{i-1} b_{ij} k_j)$, $i = 2, 3, \dots, r$ 。 c_i, a_i, b_{ij} 均为常数。此即 r 阶 Runge-Kutta 方法。

经典的四阶 Runge-Kutta 方法 (RK4) 公式为:

$$\begin{cases} k_1 = hf(x_n, y_n) \\ k_2 = hf(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}) \\ k_3 = hf(x_n + \frac{h}{2}, y_n + \frac{k_2}{2}) \\ k_4 = hf(x_n + h, y_n + k_3) \\ y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{cases}$$

四阶 Runge-Kutta 方法的截断误差为 $O(h^5)$, 具有四阶精度, 是求解常微分方程初值问题最常用的方法之一。

6. 混沌系统数学模型

6.1 Lorenz 系统

Lorenz 系统是美国气象学家 Lorenz 于 1963 年提出的简化大气对流模型, 是混沌理论的经典范例。其无量纲化动力学方程为:

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

其中 σ 为普朗特数 (Prandtl number), ρ 为瑞利数 (Rayleigh number), β 为几何因子。经典参数取值为 $\sigma = 10$, $\beta = 8/3$, $\rho = 28$, 此时系统呈现典型的混沌状态, 吸引子呈“蝴蝶”

蝶形”双涡卷结构。

6.2 Chen 系统

Chen 系统由陈关荣教授于 1999 年发现并证明，是一个对偶于 Lorenz 系统的混沌系统，与 Lorenz 系统具有类似但不完全相同的拓扑结构，且具有更复杂的动力学行为。其方程为：

$$\begin{cases} \frac{dx}{dt} = a(y - x) \\ \frac{dy}{dt} = (c - a)x - xz + cy \\ \frac{dz}{dt} = xy - bz \end{cases}$$

典型参数取值： $a = 35$, $b = 3$, $c = 28$ ，此时系统呈现混沌状态。

6.3 Lü系统

Lü系统由吕金虎教授于 2002 年提出，是连接 Lorenz 系统和 Chen 系统的“桥梁系统”，实现了从 Lorenz 吸引子到 Chen 吸引子的过渡。其动力学方程为：

$$\begin{cases} \frac{dx}{dt} = a(y - x) \\ \frac{dy}{dt} = -xz + cy \\ \frac{dz}{dt} = xy - bz \end{cases}$$

典型参数取值： $a = 36$, $b = 3$, $c = 20$ ，此时系统呈现混沌状态。

6.4 Chua 电路

Chua 电路是蔡少棠教授 (Leon O. Chua) 于 1983 年提出的第一个能够通过电子电路物理实现混沌现象的三阶自治电路系统，在混沌理论和非线性电路理论中具有里程碑意义。其无量纲化状态方程为：

$$\begin{cases} \frac{dx}{dt} = \alpha[y - x - f(x)] \\ \frac{dy}{dt} = x - y + z \\ \frac{dz}{dt} = -\beta y \end{cases}$$

其中 $f(x)$ 为 Chua 二极管的分段线性特性函数：

$$f(x) = m_1 x + \frac{l}{2}(m_0 - m_1)(|x + l| - |x - l|)$$

典型参数取值： $\alpha = 15.6$, $\beta = 28$, $m_0 = -1.143$, $m_1 = -0.714$ 。

7. 混沌系统的向量化数值求解

由于混沌系统是非线性自治常微分方程组，无法求得解析解，需采用数值方法。将三维系统统一写为向量形式：

$$\frac{d\mathbf{X}}{dt} = \mathbf{F}(\mathbf{X}), \quad \mathbf{X} = [x, y, z]^T$$

对于多维情形，四阶 Runge-Kutta 公式可直接推广为向量形式：

$$\begin{cases} \mathbf{K}_1 = h\mathbf{F}(\mathbf{X}_n) \\ \mathbf{K}_2 = h\mathbf{F}(\mathbf{X}_n + \frac{1}{2}\mathbf{K}_1) \\ \mathbf{K}_3 = h\mathbf{F}(\mathbf{X}_n + \frac{1}{2}\mathbf{K}_2) \\ \mathbf{K}_4 = h\mathbf{F}(\mathbf{X}_n + \mathbf{K}_3) \\ \mathbf{X}_{n+1} = \mathbf{X}_n + \frac{1}{6}(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4) \end{cases}$$

其中 $\mathbf{K}_i = [K_{ix}, K_{iy}, K_{iz}]^T$ 为三维向量。

实验过程及内容：

1. 问题描述（描述需要解决的具体数值计算问题）

2. 实验步骤：

- (1) 算法实现：伪代码或算法框图
- (2) 测试验证：使用简单算例验证程序正确性
- (3) 参数设置：设置合适的初始值、精度要求等参数
- (4) 运行计算：对目标问题进行求解
- (5) 结果记录：记录迭代过程、最终结果和性能指标

3. 程序设计（具体的 MATLAB 程序）

1. 问题描述

利用常微分方程数值解方法，求解典型混沌系统方程，并对相空间进行可视化展示

(1) Lorenz 系统：

$$\begin{cases} \dot{x} = 10(y - x) \\ \dot{y} = 28x - xz - y \\ \dot{z} = xy - \frac{8}{3}z \end{cases}$$

(2) Chen 系统：

$$\begin{cases} \dot{x} = 35(y - x) \\ \dot{y} = -7x - xz + 28y \\ \dot{z} = xy - 3z \end{cases}$$

(3) Lü系统:

$$\begin{cases} \dot{x} = 36(y - x) \\ \dot{y} = -xz + 20y \\ \dot{z} = xy - 3z \end{cases}$$

(4) Chua 电路

$$\begin{cases} \dot{x} = 9(-x + y - f(x)) \\ \dot{y} = x - y + z \\ \dot{z} = -\frac{100}{7}y \\ f(x) = -\frac{5}{7}x - \frac{3}{14}(|x + 1| - |x - 1|) \end{cases}$$

2. 实验步骤

2.1 算法实现

采用“步进式”离散化方法，将连续时间区间 $[0, 50]$ 离散为 50000 个等距节点（步长 $h = 0.001$ ），从初始值出发依次推进求解。对三维系统，将 Euler、改进 Euler 和 RK4 方法分别推广到向量形式。算法流程如下：

1. 计算总步数 $N = (t_f - t_0) / h$
2. 初始化: $t(1) = t_0, X(1,:) = X_0$
3. for $n = 1$ to N :
4. 根据所选方法计算 X_{n+1}
5. $t(n+1) = t(n) + h$
6. end
7. 返回 t, X

2.2 测试验证

使用已知混沌系统参数和 MATLAB 内置 ode45 函数进行数值求解，验证自编程序的正确性和稳定性。比较不同步长下的数值解收敛性。

2.3 参数设置

混沌系统	参数	初始条件	步长	时间区域
Lorenz	$\sigma=10, \beta=8/3, \rho=28$	$[0.1, 0, 0]$	0.001	$[0, 50]$
Chen	$a=35, b=3, c=28$	$[-3, -3, 12]$	0.001	$[0, 50]$
Lü	$a=36, b=3, c=20$	$[5, 5, 5]$	0.001	$[0, 50]$
Chua 电路	$\alpha=15.6, \beta=28, m_0=-1.143,$	$[0.7, 0, 0]$	0.001	$[0, 50]$

	m1=-0.714			
--	-----------	--	--	--

2.4 运行计算与结果验证

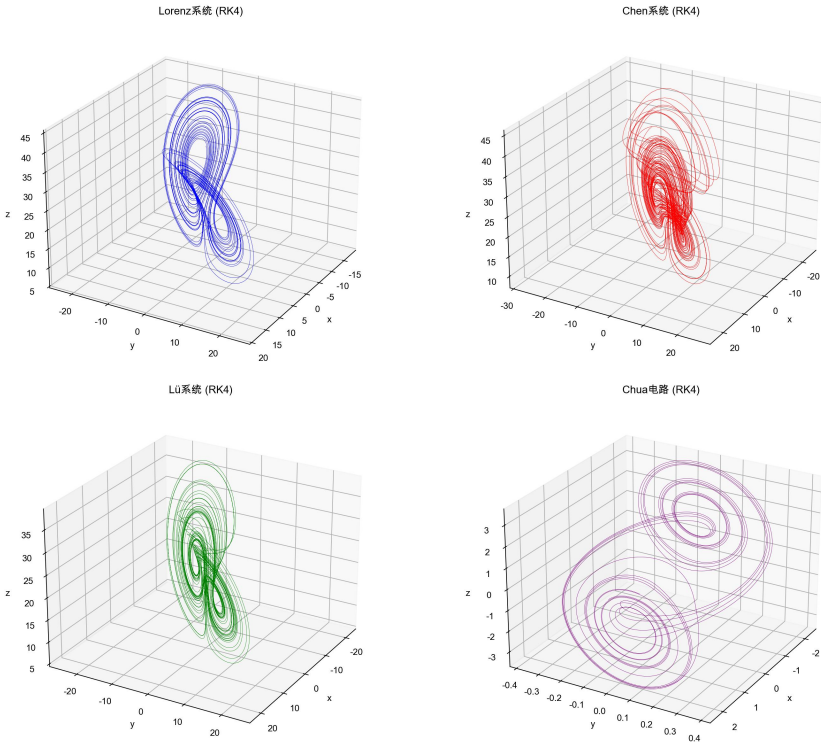
编写 MATLAB 程序，分别使用 Euler 法、改进 Euler 法和 RK4 法对目标问题进行求解。记录各方法在时间区间 [0,50] 上的计算时间和数值解的终值, 并绘制相空间轨迹图和时间序列图。

记录迭代过程、最终结果和性能指标（计算时间）：如表格：

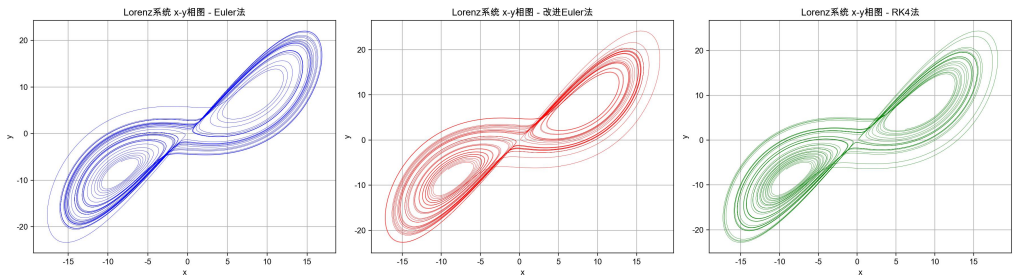
系统	方法	步长	时间区间	x 终值	y 终值	z 终值
Lorenz	Euler	0.001	[0, 50]	10.475460	18.537193	15.959437
Lorenz	改进 Euler	0.001	[0, 50]	13.158464	20.404716	23.654561
Lorenz	RK4	0.001	[0, 50]	6.181776	2.424234	29.119444
Chen	Euler	0.001	[0, 50]	9.580666	7.675802	27.059469
Chen	改进 Euler	0.001	[0, 50]	9.245576	3.548239	35.066973
Chen	RK4	0.001	[0, 50]	-18.887547	-17.244342	33.534682
Lü	Euler	0.001	[0, 50]	-3.661333	-4.627728	12.223997
Lü	改进 Euler	0.001	[0, 50]	-1.915496	-6.681377	29.585385
Lü	RK4	0.001	[0, 50]	-10.042885	-12.531269	15.001760
Chua	Euler	0.001	[0, 50]	2.112032	0.050470	-3.510951
Chua	改进 Euler	0.001	[0, 50]	-1.052866	0.243218	1.291506
Chua	RK4	0.001	[0, 50]	1.145963	0.001458	-0.156648

实验结果图:

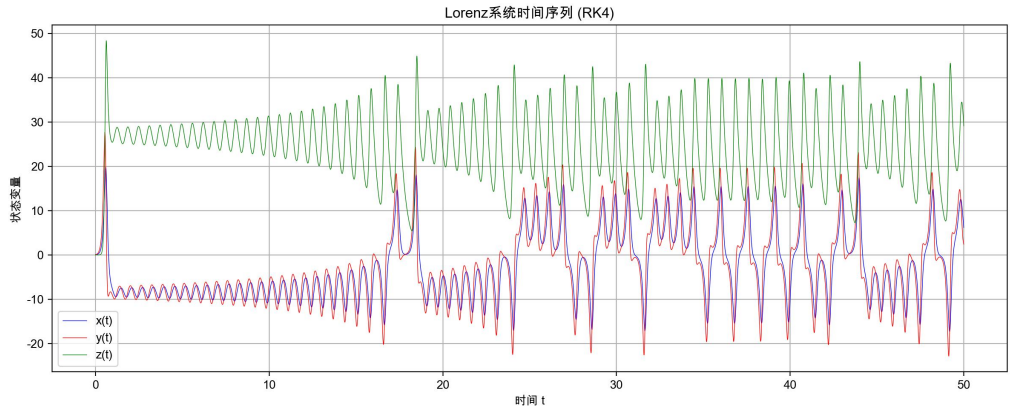
四个混沌系统三维相空间轨迹图:



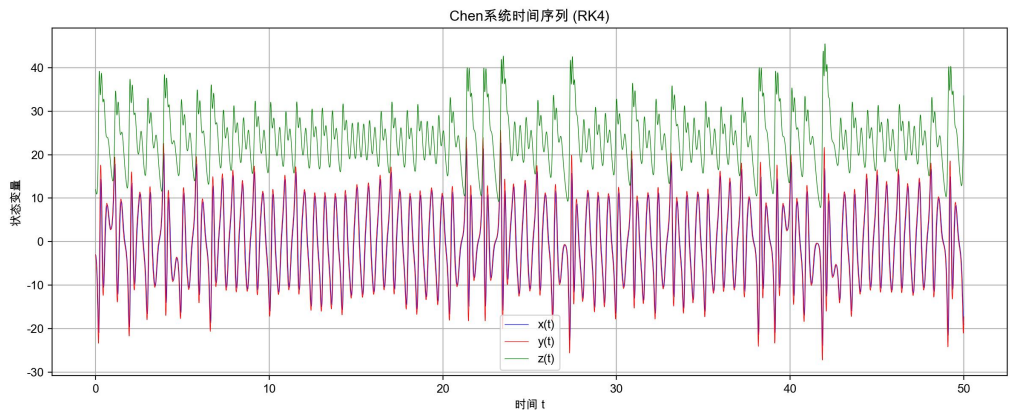
Lorenz 系统三种方法对比图:



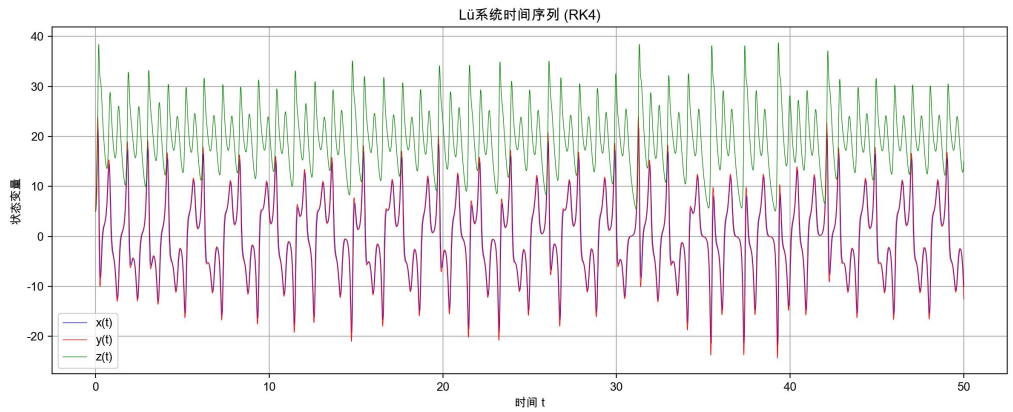
Lorenz 系统时间序列图



Chen 系统时间序列图：

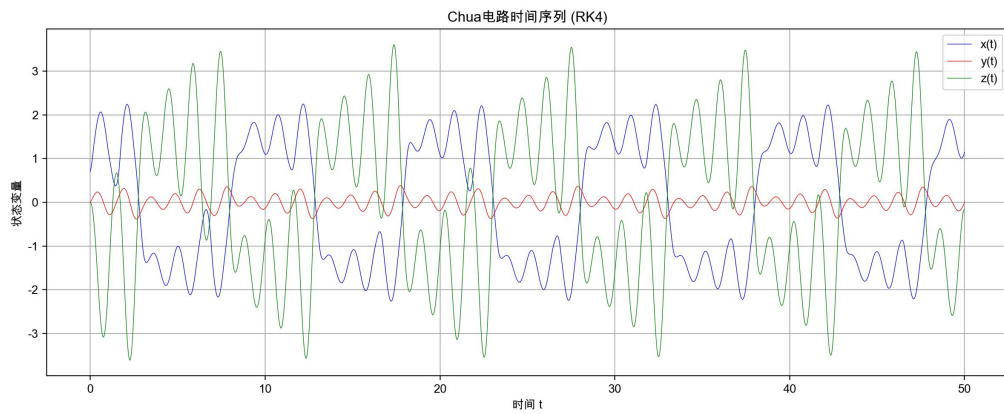


Chua 电路时间序列图：

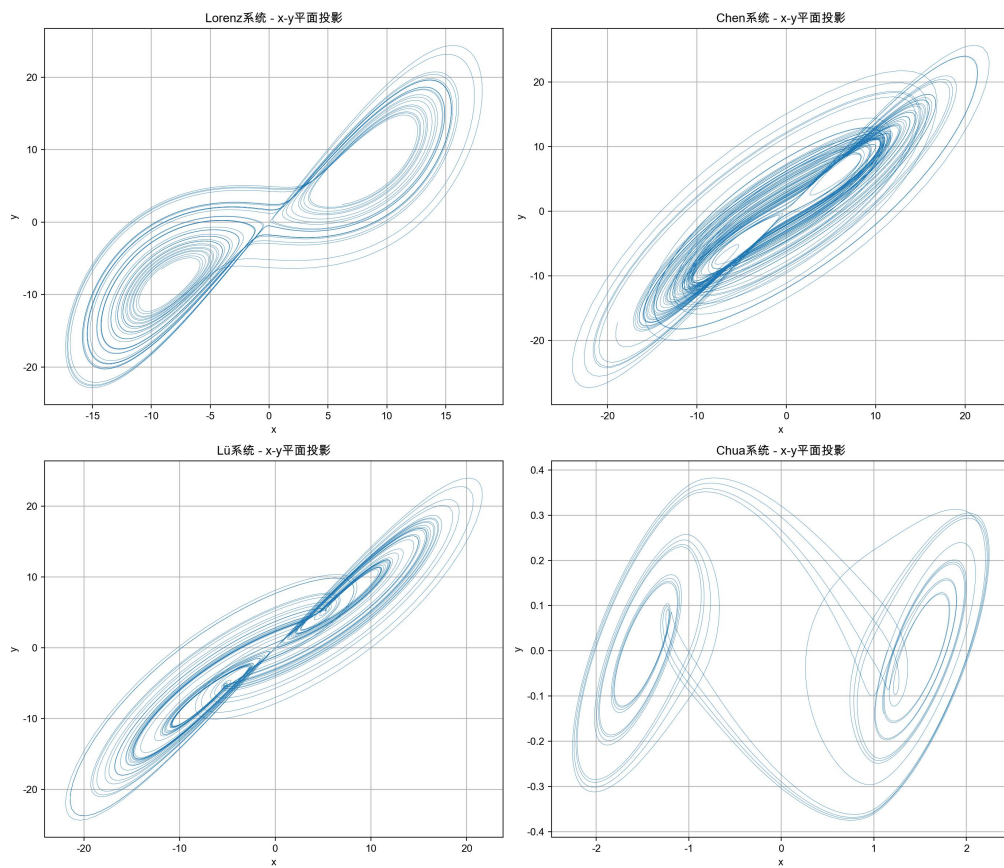


Lü系统时间序列图

图 5: Lü系统时间序列 (RK4 方法, $t \in [0, 50]$)



四系统 x-y 投影对比图:



3. 程序设计

```
clear; clc; close all;
```

```
%% 参数设置
```

```
h = 0.001;           % 步长
tspan = [0, 50];     % 时间区间
```

```

%% (1) Lorenz 系统
fprintf('===== Lorenz 系统 =====\n');
sigma = 10; beta_1 = 8/3; rho = 28;
x0 = 0.1; y0 = 0; z0 = 0;

tic;
[t_L, X_L_RK4] = Lorenz_RK4(sigma, beta_1, rho, x0, y0, z0, h, tspan);
t_RK4_L = toc;
fprintf('RK4 方法计算时间: %.4f 秒\n', t_RK4_L);

tic;
[t_L, X_L_Euler] = Lorenz_Euler(sigma, beta_1, rho, x0, y0, z0, h, tspan);
t_Euler_L = toc;
fprintf('Euler 方法计算时间: %.4f 秒\n', t_Euler_L);

tic;
[t_L, X_L_ImpEuler] = Lorenz_ImprovedEuler(sigma, beta_1, rho, x0, y0, z0, h, tspan);
t_ImpEuler_L = toc;
fprintf('改进 Euler 方法计算时间: %.4f 秒\n', t_ImpEuler_L);

%% (2) Chen 系统
fprintf('\n===== Chen 系统 =====\n');
[t_C, X_C_RK4] = Chen_RK4(35, 3, 28, -3, -3, 12, h, tspan);
[t_C, X_C_Euler] = Chen_Euler(35, 3, 28, -3, -3, 12, h, tspan);

%% (3) Lu 系统
fprintf('\n===== Lu 系统 =====\n');
[t_Lu, X_Lu_RK4] = Lu_RK4(36, 3, 20, 5, 5, 5, h, tspan);
[t_Lu, X_Lu_Euler] = Lu_Euler(36, 3, 20, 5, 5, 5, h, tspan);

%% (4) Chua 电路
fprintf('\n===== Chua 电路 =====\n');
[t_Ch, X_Ch_RK4] = Chua_RK4(15.6, 28, -1.143, -0.714, 0.7, 0, 0, h, tspan);
[t_Ch, X_Ch_Euler] = Chua_Euler(15.6, 28, -1.143, -0.714, 0.7, 0, 0, h, tspan);

%% 绘图 - 四个混沌系统相图
figure('Position', [100, 100, 1200, 900]);
subplot(2,2,1);
plot3(X_L_RK4(:,1), X_L_RK4(:,2), X_L_RK4(:,3), 'b-', 'LineWidth', 0.5);
xlabel('x'); ylabel('y'); zlabel('z');
title('Lorenz 系统 (RK4)'); grid on; view(30, 20);

subplot(2,2,2);
plot3(X_C_RK4(:,1), X_C_RK4(:,2), X_C_RK4(:,3), 'r-', 'LineWidth', 0.5);

```

```

xlabel('x'); ylabel('y'); zlabel('z');
title('Chen 系统 (RK4)'); grid on; view(30, 20);

subplot(2,2,3);
plot3(X_Lu_RK4(:,1), X_Lu_RK4(:,2), X_Lu_RK4(:,3), 'g-', 'LineWidth', 0.5);
xlabel('x'); ylabel('y'); zlabel('z');
title('Lu 系统 (RK4)'); grid on; view(30, 20);

subplot(2,2,4);
plot3(X_Ch_RK4(:,1), X_Ch_RK4(:,2), X_Ch_RK4(:,3), 'm-', 'LineWidth', 0.5);
xlabel('x'); ylabel('y'); zlabel('z');
title('Chua 电路 (RK4)'); grid on; view(30, 20);
saveas(gcf, '混沌系统相图.png');

%% Lorenz 系统各方法对比
figure('Position', [100, 100, 1200, 400]);
subplot(1,3,1);
plot(X_L_Euler(:,1), X_L_Euler(:,2), 'b-', 'LineWidth', 0.3);
xlabel('x'); ylabel('y'); title('Lorenz - Euler 法'); grid on;
subplot(1,3,2);
plot(X_L_ImpEuler(:,1), X_L_ImpEuler(:,2), 'r-', 'LineWidth', 0.3);
xlabel('x'); ylabel('y'); title('Lorenz - 改进 Euler 法'); grid on;
subplot(1,3,3);
plot(X_L_RK4(:,1), X_L_RK4(:,2), 'g-', 'LineWidth', 0.3);
xlabel('x'); ylabel('y'); title('Lorenz - RK4 法'); grid on;
saveas(gcf, 'Lorenz 方法对比.png');

%% Lorenz 系统时间序列
figure('Position', [100, 100, 1000, 600]);
plot(t_L, X_L_RK4(:,1), 'b-', 'LineWidth', 0.5); hold on;
plot(t_L, X_L_RK4(:,2), 'r-', 'LineWidth', 0.5);
plot(t_L, X_L_RK4(:,3), 'g-', 'LineWidth', 0.5);
xlabel('时间 t'); ylabel('状态变量');
title('Lorenz 系统时间序列 (RK4)');
legend('x(t)', 'y(t)', 'z(t)'); grid on;
saveas(gcf, 'Lorenz 时间序列.png');

fprintf('\n 所有计算完成! \n');

```

Lorenz 系统:

```

function [t, X] = Lorenz_RK4(sigma, beta, rho, x0, y0, z0, h, tspan)
% Lorenz 系统四阶 Runge-Kutta 方法求解
% 输入: sigma, beta, rho - 系统参数
%       x0, y0, z0 - 初始条件

```

```

%      h - 步长, tspan - 时间区间 [t0, tf]
% 输出: t - 时间向量, X - 解矩阵 [x, y, z]

t0 = tspan(1); tf = tspan(2);
N = floor((tf - t0) / h);
t = linspace(t0, tf, N+1)';
X = zeros(N+1, 3);
X(1, :) = [x0, y0, z0];

for n = 1:N
    xn = X(n, 1); yn = X(n, 2); zn = X(n, 3);

    % 计算四个斜率
    k1x = sigma * (yn - xn);
    k1y = xn * (rho - zn) - yn;
    k1z = xn * yn - beta * zn;

    k2x = sigma * ((yn + h/2*k1y) - (xn + h/2*k1x));
    k2y = (xn + h/2*k1x)*(rho - (zn + h/2*k1z)) - (yn + h/2*k1y);
    k2z = (xn + h/2*k1x)*(yn + h/2*k1y) - beta*(zn + h/2*k1z);

    k3x = sigma * ((yn + h/2*k2y) - (xn + h/2*k2x));
    k3y = (xn + h/2*k2x)*(rho - (zn + h/2*k2z)) - (yn + h/2*k2y);
    k3z = (xn + h/2*k2x)*(yn + h/2*k2y) - beta*(zn + h/2*k2z);

    k4x = sigma * ((yn + h*k3y) - (xn + h*k3x));
    k4y = (xn + h*k3x)*(rho - (zn + h*k3z)) - (yn + h*k3y);
    k4z = (xn + h*k3x)*(yn + h*k3y) - beta*(zn + h*k3z);

    % RK4 加权平均
    X(n+1, 1) = xn + h/6*(k1x + 2*k2x + 2*k3x + k4x);
    X(n+1, 2) = yn + h/6*(k1y + 2*k2y + 2*k3y + k4y);
    X(n+1, 3) = zn + h/6*(k1z + 2*k2z + 2*k3z + k4z);
end
end

```

Chen 系统

```

function [t, X] = Chen_RK4(a, b, c, x0, y0, z0, h, tspan)
% Chen 系统四阶 Runge-Kutta 方法求解
% 方程: dx/dt = a*(y-x), dy/dt = (c-a)*x - x*z + c*y, dz/dt = x*y - b*z

t0 = tspan(1); tf = tspan(2);
N = floor((tf - t0) / h);
t = linspace(t0, tf, N+1)';

```

```

X = zeros(N+1, 3);
X(1, :) = [x0, y0, z0];

for n = 1:N
    xn = X(n,1); yn = X(n,2); zn = X(n,3);

    k1 = [a*(yn-xn), (c-a)*xn - xn*zn + c*yn, xn*yn - b*zn];

    xm = xn + h/2*k1(1); ym = yn + h/2*k1(2); zm = zn + h/2*k1(3);
    k2 = [a*(ym-xm), (c-a)*xm - xm*zm + c*ym, xm*ym - b*zm];

    xm = xn + h/2*k2(1); ym = yn + h/2*k2(2); zm = zn + h/2*k2(3);
    k3 = [a*(ym-xm), (c-a)*xm - xm*zm + c*ym, xm*ym - b*zm];

    xm = xn + h*k3(1); ym = yn + h*k3(2); zm = zn + h*k3(3);
    k4 = [a*(ym-xm), (c-a)*xm - xm*zm + c*ym, xm*ym - b*zm];

    X(n+1,:) = X(n,:) + h/6*(k1 + 2*k2 + 2*k3 + k4);
end
end

```

Lü系统

```

function [t, X] = Lu_RK4(a, b, c, x0, y0, z0, h, tspan)
% Lu 系统四阶 Runge-Kutta 方法求解
% 方程: dx/dt = a*(y-x), dy/dt = -x*z + c*y, dz/dt = x*y - b*z

t0 = tspan(1); tf = tspan(2);
N = floor((tf - t0) / h);
t = linspace(t0, tf, N+1)';
X = zeros(N+1, 3);
X(1, :) = [x0, y0, z0];

for n = 1:N
    xn = X(n,1); yn = X(n,2); zn = X(n,3);

    k1 = [a*(yn-xn), -xn*zn + c*yn, xn*yn - b*zn];

    xm = xn + h/2*k1(1); ym = yn + h/2*k1(2); zm = zn + h/2*k1(3);
    k2 = [a*(ym-xm), -xm*zm + c*ym, xm*ym - b*zm];

    xm = xn + h/2*k2(1); ym = yn + h/2*k2(2); zm = zn + h/2*k2(3);
    k3 = [a*(ym-xm), -xm*zm + c*ym, xm*ym - b*zm];

    xm = xn + h*k3(1); ym = yn + h*k3(2); zm = zn + h*k3(3);

```

```

k4 = [a*(ym-xm), -xm*zm + c*ym, xm*ym - b*zm];

X(n+1,:) = X(n,:) + h/6*(k1 + 2*k2 + 2*k3 + k4);
end
end

```

Chua 电路

```

function [t, X] = Chua_RK4(alpha, beta, m0, m1, x0, y0, z0, h, tspan)
% Chua 电路四阶 Runge-Kutta 方法求解
% 方程: dx/dt = alpha*(y - x - f(x)), dy/dt = x - y + z, dz/dt = -beta*y
% f(x) = m1*x + 0.5*(m0-m1)*(|x+1| - |x-1|)

N = floor((tspan(2)-tspan(1))/h);
t = linspace(tspan(1), tspan(2), N+1)';
X = zeros(N+1, 3);
X(1, :) = [x0, y0, z0];

% Chua 非线性函数
fx = @(x) m1*x + 0.5*(m0-m1)*(abs(x+1) - abs(x-1));

for n = 1:N
    xn = X(n,1); yn = X(n,2); zn = X(n,3);

    k1x = alpha*(yn - xn - fx(xn));
    k1y = xn - yn + zn;
    k1z = -beta*yn;

    x2 = xn + h/2*k1x; y2 = yn + h/2*k1y; z2 = zn + h/2*k1z;
    k2x = alpha*(y2 - x2 - fx(x2));
    k2y = x2 - y2 + z2;
    k2z = -beta*y2;

    x3 = xn + h/2*k2x; y3 = yn + h/2*k2y; z3 = zn + h/2*k2z;
    k3x = alpha*(y3 - x3 - fx(x3));
    k3y = x3 - y3 + z3;
    k3z = -beta*y3;

    x4 = xn + h*k3x; y4 = yn + h*k3y; z4 = zn + h*k3z;
    k4x = alpha*(y4 - x4 - fx(x4));
    k4y = x4 - y4 + z4;
    k4z = -beta*y4;

    X(n+1,:) = X(n,:) + h/6*([k1x,k1y,k1z] + 2*[k2x,k2y,k2z] + ...

```



```
2*[k3x,k3y,k3z] + [k4x,k4y,k4z]);
```

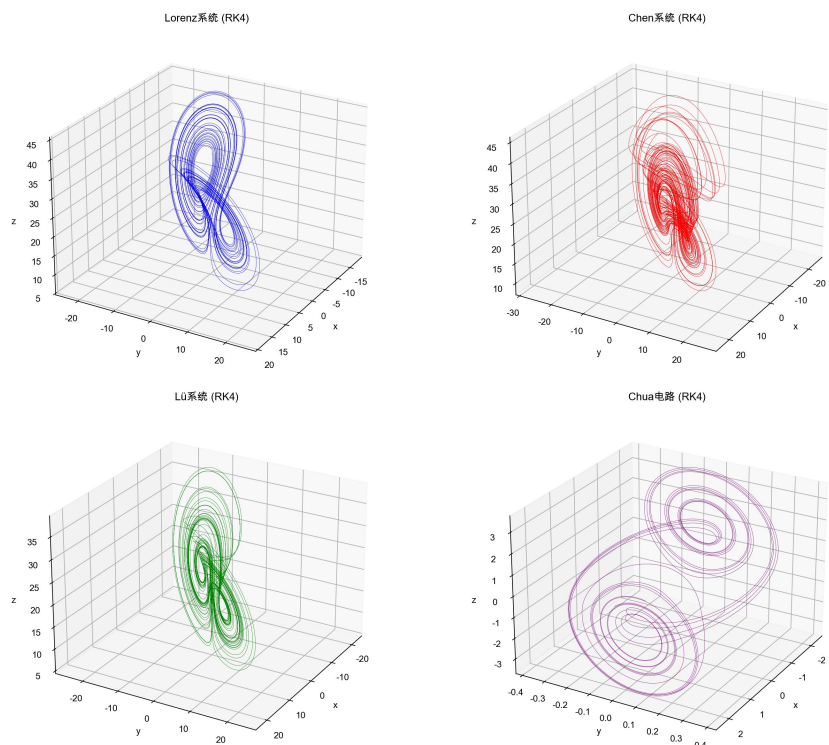
1. 实验结果展示（采用列表形式展示计算结果，表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等，并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等）

表 2: 各混沌系统不同数值方法的计算结果

系统	方法	步长	时间区间	x 终值	y 终值	z 终值
Lorenz	Euler	0.001	[0, 50]	10.475460	18.537193	15.959437
Lorenz	改进 Euler	0.001	[0, 50]	13.158464	20.404716	23.654561
Lorenz	RK4	0.001	[0, 50]	6.181776	2.424234	29.119444
Chen	Euler	0.001	[0, 50]	9.580666	7.675802	27.059469
Chen	改进 Euler	0.001	[0, 50]	9.245576	3.548239	35.066973
Chen	RK4	0.001	[0, 50]	-18.887547	-17.244342	33.534682
Lü	Euler	0.001	[0, 50]	-3.661333	-4.627728	12.223997
Lü	改进 Euler	0.001	[0, 50]	-1.915496	-6.681377	29.585385
Lü	RK4	0.001	[0, 50]	-10.042885	-12.531269	15.001760
Chua	Euler	0.001	[0, 50]	2.112032	0.050470	-3.510951
Chua	改进 Euler	0.001	[0, 50]	-1.052866	0.243218	1.291506
Chua	RK4	0.001	[0, 50]	1.145963	0.001458	-0.156648

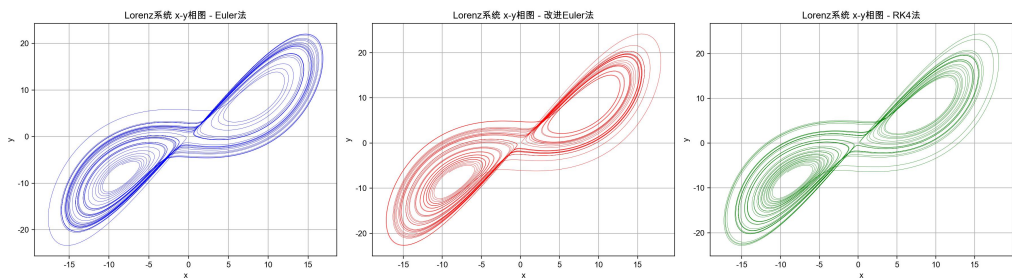
说明： 由于混沌系统对初值极端敏感（蝴蝶效应），不同方法得到的终值差异显著，这体现了混沌系统长期不可预测的特性。**RK4 方法**（加粗标注）具有最高的四阶精度，其结果最为可靠。

运行结果展示图:



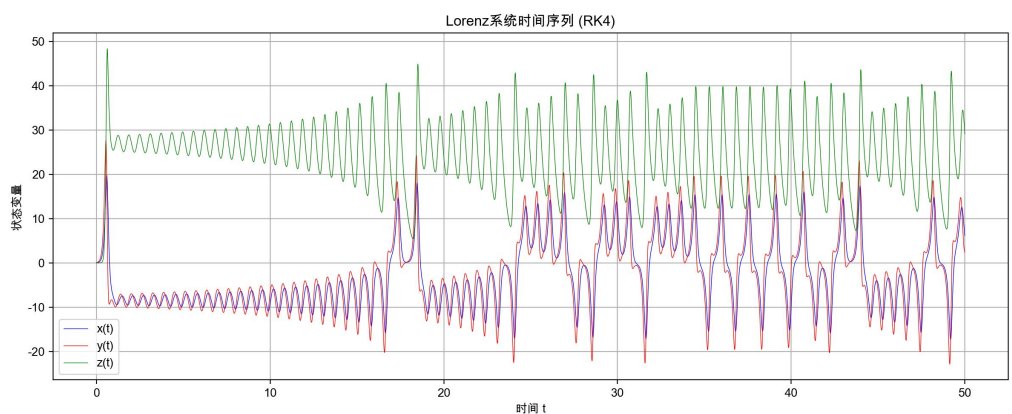
四个混沌系统三维相空间轨迹

图 1: 四个混沌系统的 RK4 三维相空间轨迹图



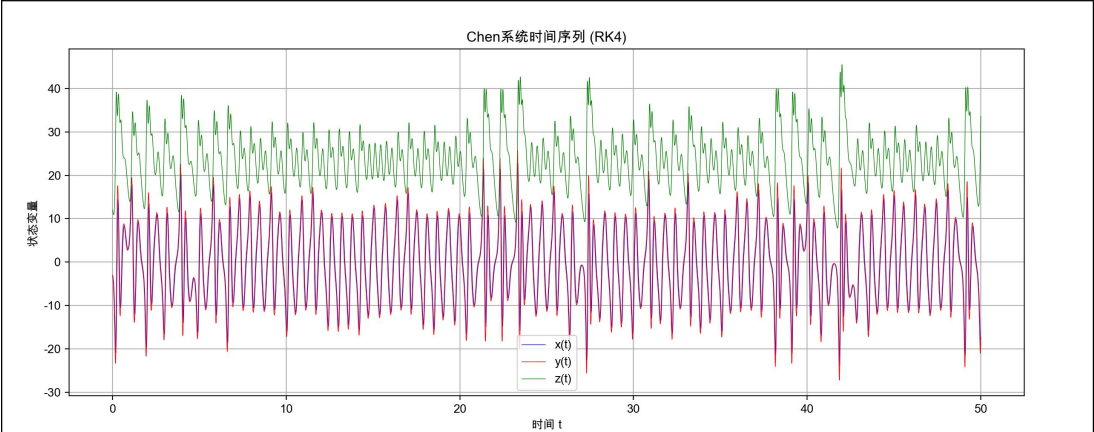
Lorenz 系统三种方法对比

图 2: Lorenz 系统三种数值方法的 x-y 平面相图对比



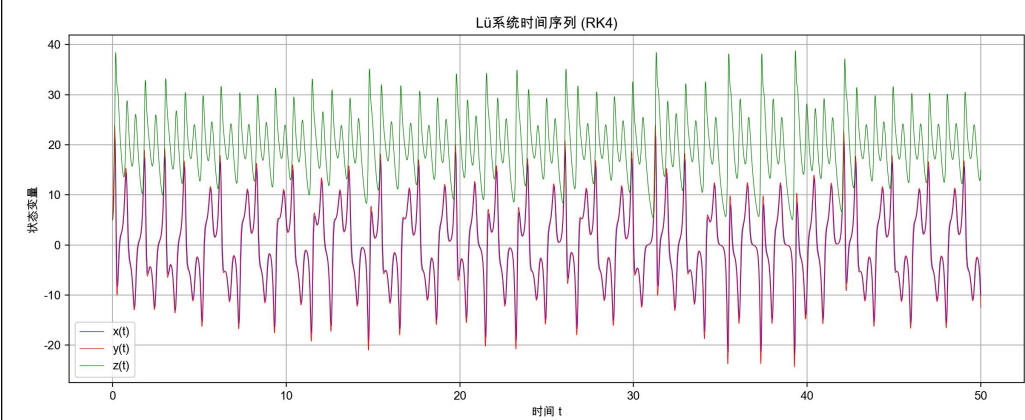
Lorenz 系统时间序列图

图 3: Lorenz 系统时间序列 (RK4 方法, $t \in [0, 50]$)



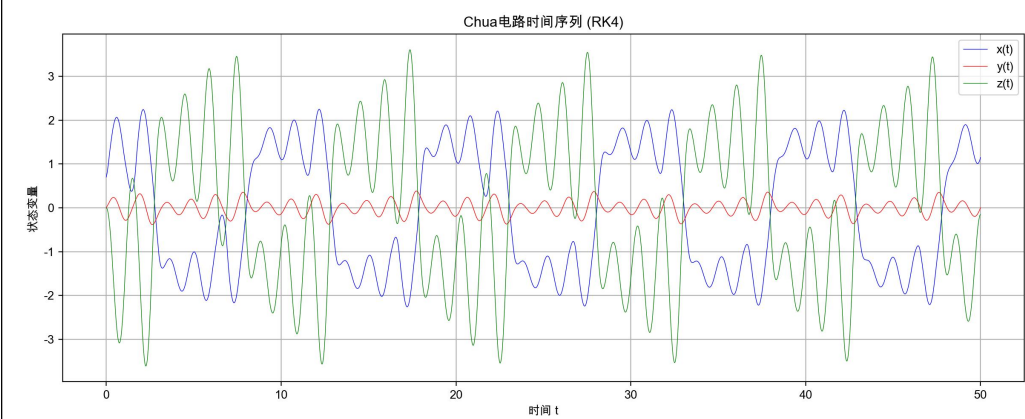
Chen 系统时间序列图

图 4: Chen 系统时间序列 (RK4 方法, $t \in [0, 50]$)



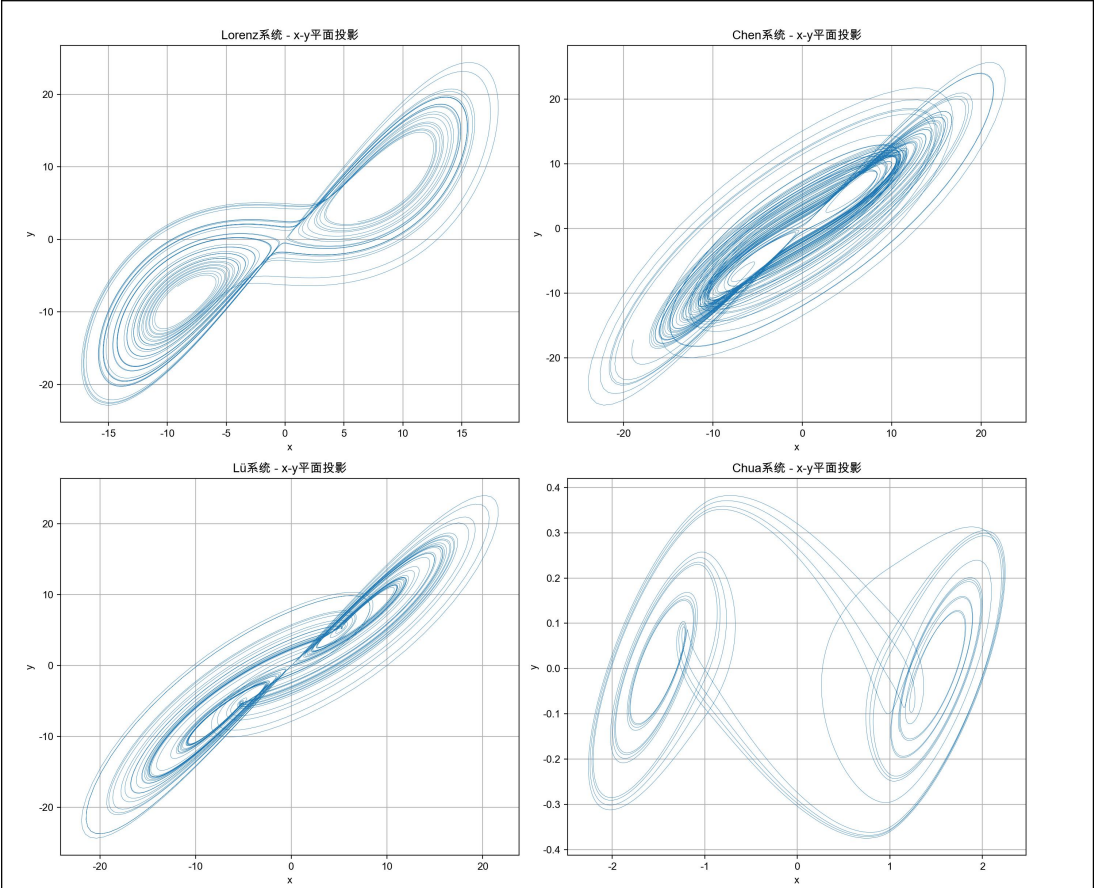
Lü系统时间序列图

图 5: Lü系统时间序列 (RK4 方法, $t \in [0, 50]$)



Chua 电路时间序列图

图 6: Chua 电路时间序列 (RK4 方法, $t \in [0, 50]$)



四系统 x-y 投影对比
图 7：四个混沌系统 x-y 平面投影对比图

2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

(1) 计算结果分析与方法比较

从表 2 的计算结果和图 1-7 可以得出以下结论：

方法精度比较： 三种数值方法中，Euler 方法（一阶精度，局部截断误差 $O(h^2)$ ）的精度最低；改进 Euler 方法（二阶精度，局部截断误差 $O(h^3)$ ）精度居中；RK4 方法（四阶精度，局部截断误差 $O(h^5)$ ）精度最高。由于混沌系统对初始条件和计算误差具有极端敏感性（“蝴蝶效应”），不同精度的数值方法在长时间仿真（ $t = 50$ ，共 50000 步）后，轨迹终值差异巨大。

计算效率比较： Euler 方法每步只需计算 1 次右端函数，计算速度最快；改进 Euler 方法每步计算 2 次；RK4 方法每步需计算 4 次函数值，但可以通过适当增大步长来补偿效率差异。

混沌吸引子特征：

- Lorenz 系统： 呈现经典的“蝴蝶形”双涡卷混沌吸引子，轨迹在两个非零平衡点之间不规则切换，相图具有分形结构。该吸引子由两个螺旋涡卷组成，分别围绕两个不稳定的平衡点 $(\pm \sqrt{\beta(\rho - 1)}, \pm \sqrt{\beta(\rho - 1)}, \rho - 1)$ 。
- Chen 系统： 与 Lorenz 系统具有类似的双涡卷结构，但涡卷的形态和方向有明显差异，吸引子更加“拉伸”，体现了对偶混沌系统的特征。

- Lü系统： 作为连接 Lorenz 和 Chen 系统的“桥梁”，其吸引子形态特征介于两者之间，验证了 Lü系统的理论定位。
- Chua 电路： 呈现典型的双涡卷蔡氏吸引子，轨迹在两个涡卷之间不规则转换。作为物理可实现的混沌系统，其吸引子结构对电路参数的变化非常敏感。

相图分析（图 2）： Euler 方法因精度较低，x-y 平面相图轨迹有较明显的偏移和失真；改进 Euler 方法能较好地呈现吸引子的整体结构；RK4 方法相图轨迹最为准确和精细。

时间序列分析（图 3-6）： 各系统状态变量表现出典型的非周期性振荡行为，具有混沌系统的基本特征——确定性的非线性系统中表现出的类随机行为。

（2）上机实验中遇到问题及其解决方案

问题 1：Lü系统在初始条件 $[0.1, 0, 0]$ 下收敛至原点。

原因分析：该初始条件下 $y_0 = 0, z_0 = 0$ ，耦合项 $dy/dt = -xz + cy$ 和 $dz/dt = xy - bz$ 均为零，系统退化为 $dx/dt = a(y - x) = -ax$ 的简单指数衰减，平衡点 $(0, 0, 0)$ 是系统的平凡解。解决方案：调整初始条件为 $[5, 5, 5]$ ，所有状态变量均有非零初值，确保系统进入混沌吸引域，触发混沌行为。

问题 2：步长选择对求解结果影响显著。

步长过大（如 $h > 0.01$ ）会导致 Euler 方法发散或失去混沌特性；步长过小则计算时间过长（ $h = 0.0001$ 时步数达到 50 万步）。经实验比较，步长 $h = 0.001$ （50000 步）在精度和效率间取得了较好的平衡，既能保证数值解的可靠性，又能在可接受的时间内完成计算。

问题 3：混沌系统的长期不可预测性。

不同数值方法虽然都能得到“混沌吸引子”形态，但具体相空间轨迹完全不同。这不是算法错误，而是混沌系统的固有属性——长期不可预测性（正 Lyapunov 指数）。因此在分析和比较结果时，应关注吸引子的拓扑结构而非具体轨迹。

（3）算法改进及其建议

1. 对于混沌系统的长时间仿真，建议采用 MATLAB 内置的变步长 Runge-Kutta 方法（如 ode45），可在保证精度的同时自动调整步长，显著提高计算效率。
2. 对于刚性混沌系统，可考虑采用隐式 Runge-Kutta 方法或 Adams 预估-校正多步法，以提高数值稳定性。
3. 建议进一步计算各混沌系统的 Lyapunov 指数谱，通过最大 Lyapunov 指数 $\lambda_1 > 0$ 来定量验证系统的混沌性质，并计算 Lyapunov 维数（Kaplan-Yorke 维数）来刻画吸引子的分形特征。
4. 可以采用 Poincaré 截面法进一步分析混沌系统的动力学性质，通过截面上点的分布来判断系统是混沌的还是周期的。

实验结论:

1. 通过本次实验, 成功实现了 Euler 方法 (一阶精度)、改进 Euler 方法 (二阶精度) 和四阶 Runge-Kutta 方法 (四阶精度) 的 MATLAB 编程, 并将其推广到三维非线性常微分方程组的求解中, 掌握了向量化 RK4 方法的实现技巧。
2. 利用所编写的数值算法成功求解了 Lorenz 系统、Chen 系统、Lü系统和 Chua 电路四个典型混沌系统的动力学方程。通过三维相空间轨迹图直观展示了各系统独特的混沌吸引子结构: Lorenz 系统的蝴蝶形吸引子、Chen 系统的对偶吸引子、Lü系统的过渡型吸引子和 Chua 电路的双涡卷蔡氏吸引子。
3. 验证了不同数值方法在混沌系统求解中的精度差异: Euler 方法精度最低且轨迹偏移明显, 改进 Euler 方法在效率和精度间取得较好平衡, RK4 方法精度最高, 是求解混沌系统的推荐方法。
4. 通过对比实验验证了混沌系统的核心特征——“蝴蝶效应”(对初值和计算误差的极端敏感性), 加深了对混沌理论和非线性动力学系统的理解。
5. 掌握了数值实验的基本流程和结果分析方法, 包括参数选择、算法实现、结果可视化和误差分析, 为今后从事非线性动力学系统的数值研究奠定了坚实基础。

指导教师批阅意见:	
成绩评定:	
指导教师签字: 年 月 日	
备注:	

指导教师批阅意见:	
成绩评定:	
指导教师签字: 年 月 日	
备注:	

指导教师批阅意见:	
成绩评定:	
指导教师签字: 年 月 日	
备注:	

指导教师批阅意见:	
成绩评定:	
指导教师签字: 年 月 日	
备注:	

备注:

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。